

EmberDB: A FHIR-Native Time-Series Storage Engine for Continuous Patient Monitoring

Abhinav Srivastava
CTO, Ambra
Abhinav@ambra911.com

Abstract—ICU monitors sample vital signs every few seconds. The standard interoperability format for those readings is HL7 FHIR Observation. General-purpose time-series databases such as InfluxDB and TimescaleDB ingest the data but drop the clinical semantics at the storage layer, which forces reconstruction at query time. EmberDB is a Rust time-series engine for FHIR Observation data. Each record carries a composite metric key of patient identifier, LOINC code, and unit, and the data is partitioned into 1-hour chunks behind a write-ahead log. The engine includes a suite of clinical pattern-detection algorithms; we benchmark four of them in this paper: CUSUM changepoint detection, seasonal decomposition, Mahalanobis anomaly scoring, and moving-window analysis (a PELT segmentation method is implemented but not yet benchmarked). On a synthetic MIMIC-IV workload of 500 patients, 48-hour ICU stays, and 1.7M chartevents, EmberDB answers patient-scoped point lookups in single-digit microseconds. We benchmark it head to head against three baselines on the identical workload: an embedded SQLite store and two purpose-built time-series engines, InfluxDB 2.7 and TimescaleDB on PostgreSQL 16, both run in Docker. On this comparative workload EmberDB ingests $3.8\text{--}9.4\times$ faster than these systems (1.18M records/sec, against 0.12–0.31M for the baselines) and answers patient-scoped point lookups in single-digit microseconds, against hundreds to thousands of microseconds for the baselines. In a separate standalone micro-benchmark with persistence disabled, EmberDB reaches 3.2M records/sec in memory mode with sub-microsecond point queries; this is a different, no-baseline configuration and is not the figure behind the comparative ratios. The advantage narrows on cohort scans, where TimescaleDB is competitive, and InfluxDB stores the data far more compactly. Part of EmberDB’s point-query advantage over the client-server baselines reflects its in-process library path versus their network round-trip, an axis we separate in the evaluation. All results are single-node, at a single scale, and use a synthetic schema; we report the cases where EmberDB loses alongside the cases where it wins.

Index Terms—FHIR, time-series database, patient monitoring, ICU, clinical informatics

I. INTRODUCTION

ICU bedside monitors sample vital signs (heart rate, blood pressure, oxygen saturation, respiratory rate, temperature) at intervals from one second to a few minutes. Across a hospital that adds up to millions of observations per patient-day [1]. HL7 FHIR has become the standard wrapper for these readings. Each measurement is an `Observation` resource carrying a LOINC-coded type, a numeric value with units, a patient reference, and a timestamp [3].

The open problem is storing those observations efficiently without losing the clinical structure. InfluxDB [4] and TimescaleDB [5] can absorb the throughput, but neither preserves the structured relationship between patient, observation code, and clinical context. InfluxDB flattens FHIR into measurement/tag/field triples. TimescaleDB keeps the relational schema but pays row-storage and SQL-parsing overhead on every high-frequency point lookup, and plain PostgreSQL needs joins to answer temporal queries because it has no time-partitioned storage of its own.

EmberDB is a Rust time-series storage engine built around continuous patient monitoring. Its observations carry composite metric keys that encode patient ID, LOINC code, and unit, so a patient-scoped read is a prefix match and nothing has to map back to FHIR after ingestion. Data is bucketed into configurable time chunks, one hour by default, so range scans touch only the relevant chunks and retention can drop whole chunks at once. A write-ahead log with chunk-level durability markers handles crash recovery without slowing ingestion. A set of pattern-detection algorithms runs inside the engine, on the same data path as persistence. This paper benchmarks four of them (CUSUM changepoint, seasonal decomposition, multivariate Mahalanobis anomaly detection, and moving-window analysis); a PELT segmentation method is also implemented in the engine but is not benchmarked here.

We evaluate EmberDB on synthetic MIMIC-IV chartevents and benchmark it head to head against three baselines on the identical workload: SQLite, InfluxDB 2.7, and TimescaleDB on PostgreSQL 16. We report where it wins and where it loses.

II. BACKGROUND AND RELATED WORK

A. Clinical time-series data

Clinical streams differ from IoT or infrastructure metrics in ways that matter for storage. The signals are multivariate and physiologically correlated, so heart rate, blood pressure, and SpO2 do not move independently. Each value also carries semantics that have to survive end to end: a LOINC code, a patient reference, and clinical context. Sampling is irregular too, with the measurement frequency set by patient acuity and clinical protocol rather than a fixed cadence.

MIMIC-IV [2] is the reference dataset for these properties. Its `chartevents` table runs to hundreds of millions of rows

across thousands of ICU stays, with item IDs that map to specific physiological measurements.

B. Existing storage approaches

InfluxDB organizes data by measurement name with tag-based indexing, which suits infrastructure monitoring. Storing FHIR Observations means flattening each one into measurement/tag/field triples, and that discards the structured patient/code/value relationship.

TimescaleDB adds time-partitioned hypertables to PostgreSQL. The relational schema survives, but row-based storage and SQL parsing stay on the hot path for high-frequency point lookups.

SQLite is a common embedded choice for clinical applications, and its simplicity is the appeal. The cost is the absence of time-aware partitioning and the manual index management that temporal queries then require.

OpenTSDB and QuestDB are purpose-built time-series stores. Like InfluxDB, both treat every metric as a generic numeric stream with no first-class notion of a healthcare resource model.

III. SYSTEM DESIGN

A. Architecture overview

EmberDB is written in Rust and exposes both a library API and a REST interface built on `warp`. Internally it has three layers: an in-memory chunk manager, a write-ahead log for durability, and a JSON chunk persistence layer.

B. FHIR-native metric encoding

Each FHIR Observation becomes an internal `Record` with a composite metric name:

```
metric_name = "{patient_id}|{loinc_code}|{unit}"
```

A heart rate for patient P1234 ends up stored as P1234|8867-4|/min. The encoding lets the engine answer both patient-scoped and code-scoped queries by prefix matching on the key, with no secondary indexes.

Multi-component observations (blood pressure, with systolic and diastolic, is the canonical case) are decomposed into records sharing a prefix. The FHIR component structure is preserved, and each component can be analysed as its own time series.

C. Time-partitioned chunks

Records live inside `TimeChunk` structures, each spanning a configurable duration (one hour by default). A chunk contains a `HashMap<String, Vec<Record>>` mapping metric names to sorted record vectors, a resource-type index mapping FHIR resource types to their constituent metric names, and a small block of metadata (creation time, record count, dirty flag). The chunk boundary for any timestamp is computed by flooring to the nearest multiple of the chunk duration, which keeps chunk identification at $O(1)$.

D. Write-ahead log

The WAL gives EmberDB crash recovery. Each record is persisted before it lands in the in-memory chunk. Records are serialised as JSON with a 4-byte length prefix. A chunk that crosses 10,000 records or 1 MB is flushed, and the matching WAL entries are marked durable at that point. On recovery the engine loads the persisted chunks from disk and replays whatever WAL entries remain.

E. MIMIC-IV data loader

A MIMIC-IV loader module ships with EmberDB. It maps chartevent item IDs to FHIR LOINC codes (itemid 220045 $\rightarrow$ LOINC 8867-4 for heart rate, and so on) and turns rows into `Record` structures directly. The path is zero-copy, so there is no overhead from building intermediate FHIR JSON.

IV. PATTERN DETECTION

EmberDB provides built-in pattern detection algorithms, configured via a TOML file and operating directly on stored records. We describe five algorithms below and report measured latency for four of them in Table III; the PELT segmentation method is implemented but, as it has not yet been benchmarked under the same harness, is omitted from the latency table and excluded from the head count of benchmarked algorithms.

A. CUSUM changepoint detection

The Cumulative Sum (CUSUM) algorithm maintains running sums S^+ and S^- tracking positive and negative deviations from the series mean:

$$S_i^+ = \max(0, S_{i-1}^+ + (x_i - \mu - k))$$

$$S_i^- = \max(0, S_{i-1}^- + (\mu - k - x_i))$$

where $k = 0.5\sigma$ is the sensitivity parameter. A changepoint is declared when either sum exceeds the threshold $h = \theta\sigma$, where θ is configurable (default 2.0).

B. PELT segmentation

The Pruned Exact Linear Time (PELT) algorithm finds the optimal segmentation by minimizing the penalized cost:

$$\sum_{j=1}^{m+1} \mathcal{C}(y_{\tau_{j-1}+1:\tau_j}) + m \cdot \beta$$

where $\mathcal{C}$ is the Gaussian negative log-likelihood cost for each segment and β is the penalty term. Changepoints with magnitude below $\theta\sigma$ are pruned. This method is implemented in the engine but is not included in the latency benchmark of Table III.

C. Seasonal decomposition

Time series are decomposed into trend T_t , seasonal S_t , and residual R_t components using moving-average smoothing. Both additive ($Y_t = T_t + S_t + R_t$) and multiplicative ($Y_t = T_t \times S_t \times R_t$) models are supported.

D. Mahalanobis anomaly detection

For multivariate anomaly detection across correlated vitals, EmberDB computes the Mahalanobis distance:

$$D_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$

Points exceeding a configurable threshold (default 3.0) are flagged as outliers. Metric groups can be specified explicitly or auto-detected via Pearson correlation.

E. Moving window analysis

A sliding window computes per-window statistics (trend slope, volatility, or range) and flags windows whose statistic deviates from the population by more than θ standard deviations.

V. EVALUATION

We evaluate EmberDB on three benchmark suites: standalone performance, comparative analysis against SQLite, and a synthetic MIMIC-IV workload run head to head against three baselines. All experiments were run on a single Apple Silicon machine running macOS, with EmberDB compiled in release mode (`--release`). The two purpose-built baselines run in Docker on the same host: InfluxDB 2.7 (write via line protocol over HTTP, queries in Flux) and TimescaleDB on PostgreSQL 16 (bulk load via `COPY` into a one-hour hypertable, queries in SQL over a composite (`subject_id`, `itemid`, `charttime`) index). SQLite uses WAL mode with `PRAGMA synchronous=NORMAL`. Every system ingests the same generated dataset (seed 42) and answers the same four clinical query shapes, so the numbers are comparable. The baseline harness is `benches/baseline_bench.rs`, and every figure is recorded in `head_to_head_results.csv`.

Measurement methodology and its limits. The harness reports the arithmetic mean latency over a fixed number of repetitions per query (200 for EmberDB, SQLite, and TimescaleDB; 10–50 for the slower InfluxDB queries to bound wall-clock time). It does not currently record per-iteration samples, so we report means only and do *not* report median, p99, or a dispersion measure; nor do we apply an explicit warmup-discard step. The microsecond-scale deltas between EmberDB and SQLite on point queries should therefore be read as order-of-magnitude indicators rather than statistically tested differences, and the lower InfluxDB iteration count makes its means correspondingly noisier. Adding an equal, larger iteration budget with warmup discard and full latency distributions is the obvious next hardening step and is left to future work.

In-process versus client-server paths. The four systems do not share a client path, and this matters for interpreting the query numbers. An EmberDB query is an in-process library call with no serialization or transport. A TimescaleDB query crosses a local PostgreSQL socket and is parsed and planned as SQL. An InfluxDB query is an HTTP round-trip whose body is a Flux program compiled on every invocation. A

meaningful share of the InfluxDB gap in particular — its 4,949 μs single-vital point query against EmberDB’s 4.0 μs — is therefore transport and query-compilation overhead inherent to a client-server engine, not storage-layer cost, and is not the same axis of comparison as EmberDB’s embedded design. We did not instrument the engines to separate the transport component numerically; readers should attribute the largest InfluxDB deltas partly to the client path rather than entirely to the storage engine.

A. Standalone performance

1) *Write throughput:* Table I shows write throughput across batch sizes with persistence disabled (memory-optimized mode).

TABLE I
EMBERDB WRITE THROUGHPUT (MEMORY MODE)

Records	Throughput (rec/s)
1,000	2,689,372
10,000	1,467,926
50,000	2,582,106
100,000	3,238,495
500,000	3,297,006

2) *Query latency:* Table II reports average query latency over 100K ingested records (100 patients, single metric).

TABLE II
EMBERDB QUERY LATENCY (μs)

Query Type	Avg Latency (μs)
Point (narrow range)	0.1
Latest value	0.9
Patient + code	126.2
Full patient (prefix)	13.6
Time range (50K span)	61.1

3) *Pattern detection:* Table III reports per-invocation latency for each detection algorithm on 2,000 records.

TABLE III
PATTERN DETECTION LATENCY (μs)

Algorithm	Avg Latency (μs)
CUSUM changepoint	598.0
Seasonal decomposition	5,167.5
Moving window	423.5
Mahalanobis (multivariate)	251.5

4) *WAL performance:* With persistence enabled (`fsync` per record), EmberDB sustains 124 records/s write throughput due to disk synchronization overhead. Recovery from WAL (10,000 records) completes in 0.008 seconds.

5) *Storage overhead:* At 100,000 records with full persistence, EmberDB uses 149.6 bytes per record on disk (14.96 MB total) due to JSON serialization overhead. This standalone figure is measured on a uniform 100K-record single-metric dataset by `benches/eval.rs`.

The head-to-head table below (Table VI) reports a higher 201.4 bytes/record for the same JSON chunk format, because it is measured by a separate probe (benches/ember_storage_probe.rs) on the full 1.728M-record MIMIC workload, which spans 500 patients and 6 vitals across many one-hour chunks. The larger figure reflects the per-chunk and per-metric-key overhead that grows with the number of distinct `patient|code|unit` keys and time chunks, which the single-metric micro-benchmark does not exercise; the 201.4 B/rec measurement is the one to use for the cross-system comparison.

B. Comparative analysis: EmberDB vs. SQLite

1) *Write throughput*: Table IV compares write throughput. EmberDB’s in-memory mode avoids disk synchronization overhead, while SQLite uses WAL mode with `PRAGMA synchronous=NORMAL`.

TABLE IV
WRITE THROUGHPUT COMPARISON (REC/S)

Records	EmberDB	SQLite	Ratio
1,000	738,257	246,038	3.0×
10,000	1,258,139	247,915	5.1×
50,000	2,145,535	282,572	7.6×
100,000	2,197,044	263,857	8.3×
500,000	2,170,299	254,397	8.5×

2) *Query latency*: On a uniform 100K-record micro-benchmark, the gap over SQLite is widest where the workload is purely patient-scoped: a point lookup runs in 0.1 μ s vs. 764.5 μ s (8,890 $\times$) and a latest-value read in 0.9 μ s vs. 857.4 μ s (953 $\times$). It narrows on scans, to 12.8 $\times$ on a time range and 7.1 $\times$ on a full-patient read, where both engines touch more records. The sub-microsecond point-query figure (0.1 μ s) is specific to this uniform standalone setting; on the more realistic MIMIC workload below the same query shape costs 4.0 μ s, and that single-digit-microsecond figure, not the sub-microsecond one, is the appropriate point of comparison against the purpose-built engines. The MIMIC results below show the same shape against those engines.

C. MIMIC-IV synthetic workload

We generated synthetic MIMIC-IV chartevents for 500 patients, each with a 48-hour ICU stay and 6 vital signs sampled every 5 minutes (576 measurements per vital per patient). Vital sign distributions were calibrated to published ICU ranges: HR $\mathcal{N}(84, 17)$, SBP $\mathcal{N}(121, 23)$, DBP $\mathcal{N}(70, 14)$, SpO2 $\mathcal{N}(96.8, 2.8)$, RR $\mathcal{N}(18, 4)$, Temp $\mathcal{N}(98.6, 1.2)$ °F. Each chartevent was mapped to a FHIR Observation via LOINC code and ingested into both EmberDB and SQLite.

1) *Ingestion*: The dataset contains 1,728,000 chartevents (500 patients $\times$ 6 vitals $\times$ 576 measurements). EmberDB ingested the FHIR-converted records at 1,176,849 records/sec (1.47s total). On the same data, SQLite reached 124,863 records/sec, TimescaleDB 309,707 records/sec via `COPY`, and InfluxDB 287,412 records/sec via the line protocol. EmberDB

is 9.4 $\times$ faster than SQLite and 3.8 $\times$ faster than TimescaleDB, the fastest of the purpose-built baselines on ingestion. This 1.18M records/sec is the comparative, apples-to-apples figure against which the 3.8–9.4 $\times$ ratios are computed. It is lower than the 3.2M records/sec of the standalone memory-mode micro-benchmark in Table I because that micro-benchmark runs with persistence disabled and no baseline alongside it; the two figures describe different configurations and should not be conflated.

2) *Query performance*: Table V reports query latency for the same four query shapes on each system.

TABLE V
MIMIC-IV QUERY LATENCY (μ S), EMBERDB VS. SQLITE

Query	EmberDB	SQLite	Speedup
Single vital (1h)	4.0	37.1	9.3×
Full patient stay	1,145.9	746.9	0.65×
Cohort vital (1h)	967.9	21,191.0	21.9×
Latest vital	31.3	96.5	3.1×

3) *Head-to-head against purpose-built engines*: Table VI places all four systems side by side on the MIMIC workload: write throughput, the four query latencies, and on-disk storage. Lower is better for every row except ingestion.

TABLE VI
HEAD-TO-HEAD ON THE SYNTHETIC MIMIC-IV WORKLOAD (1.728M RECORDS). INGESTION IN RECORDS/SEC (HIGHER IS BETTER); QUERY LATENCY IN μ S AND STORAGE IN BYTES/RECORD (LOWER IS BETTER).

Metric	EmberDB	SQLite	TimescaleDB	InfluxDB
Ingest (rec/s)	1,176,849	124,863	309,707	287,412
Single vital (1h)	4.0	37.1	561.6	4,949
Full patient stay	1,145.9	746.9	3,628	13,913
Cohort vital (1h)	967.9	21,191	847.3	12,692
Latest vital	31.3	96.5	2,466	4,618
Storage (B/rec)	201.4	103.1	129.6	26.8

The two patient-scoped point queries follow the same pattern. EmberDB wins ingestion and both the single-vital and latest-vital lookups by one to three orders of magnitude over the purpose-built engines, because each reduces to a prefix match on an in-memory chunk with no SQL parsing. The single-vital lookup runs 140 $\times$ faster than TimescaleDB and over 1,200 $\times$ faster than InfluxDB. These large multiples should be read with the client-path caveat above: EmberDB’s lookup is an in-process call, while the TimescaleDB and especially the InfluxDB figures include socket or HTTP transport and, for InfluxDB, Flux compilation on every iteration, so part of the gap is the embedded-versus-client-server axis rather than the storage engine alone. TimescaleDB is the strongest baseline, and it wins the cohort scan: its item-indexed hypertable answers a one-shot range query while EmberDB iterates 500 per-patient metrics. InfluxDB is slowest on every query in this point/range mix, which suits its tag-cardinality model poorly — though this mix omits the time-bucketed downsampling and multi-tag aggregation queries that

InfluxDB and TimescaleDB are built for, so the comparison understates them (see the scope discussion below). InfluxDB does store the data more compactly: TSM columnar compression yields 26.8 bytes/record against EmberDB’s 201.4, a $7.5\times$ gap. The JSON chunk format is what pays for that gap, and the full-patient-stay query remains EmberDB’s weakest case for the reason given in the SQLite comparison.

VI. DISCUSSION

FHIR-native advantages. By encoding patient ID and LOINC code directly into the metric key, EmberDB removes the need for secondary indexes or joins when retrieving patient-scoped observations. The design fits clinical dashboard queries that retrieve all vitals for a single patient over a shift or stay.

Pattern detection at the storage layer. Embedding change-point detection and anomaly scoring directly in the storage engine avoids data export overhead. Real-time alerting pipelines can operate on the same data path as persistence.

Where EmberDB loses. The JSON chunk format is the main weakness. On the MIMIC workload it costs 201.4 bytes/record on disk, against 129.6 for TimescaleDB, 103.1 for SQLite, and 26.8 for InfluxDB, whose TSM columnar compression is $7.5\times$ more compact. Two query shapes also go against EmberDB. On the full-patient-stay query SQLite wins (747 μ s vs. 1,146 μ s) because EmberDB issues a separate range query per vital metric while SQLite returns every vital in one index scan. On the cohort scan TimescaleDB wins (847 μ s vs. 968 μ s) for the same structural reason: its item-indexed hypertable answers the window in a single query rather than 500 per-patient lookups. Both follow from the prefix-keyed layout, and a columnar chunk format with a multi-metric range API would address them.

Other limitations. The single-writer architecture limits concurrent write throughput. The pattern-detection algorithms use simplified implementations; a production deployment would need validated statistical libraries.

Scope of the evaluation. Every measurement here is single-node, at a single scale, and uses a synthetic schema. The entire cross-system comparison is one 1.728M-record point at one cardinality on one Apple Silicon host; we have not run it at a second scale or on real MIMIC-IV `chartevents`, so claims that the ordering generalizes are hypotheses, not results. The workload is generated from published ICU vital-sign distributions rather than real `chartevents`, and all four systems ran on one machine, so the numbers capture single-host engine behavior, not clustered or networked deployment. Because the systems are exercised through different client paths — an in-process call for EmberDB versus a socket or HTTP round-trip for the server engines — the query-latency multiples conflate storage-engine cost with transport and query-compilation overhead, and we did not separate the two numerically. The point/range query mix also favors a prefix-keyed store and omits the analytical scans (time-bucketed downsampling, multi-tag aggregation, gap-fill) that InfluxDB and TimescaleDB are tuned for; adding one or

two such queries is needed before the head-to-head can be called balanced, and we flag the current mix as tilted toward EmberDB. We therefore frame the relative ordering as a hypothesis to be confirmed: a second scale, a real-data run, and the missing analytical query shapes would each de-risk it, and the absolute figures and the storage gap in particular would shift with a columnar format and a real workload.

Future work. Planned improvements include columnar chunk encoding (e.g., Parquet), concurrent chunk writers with sharded locking, integration with FHIR Subscription for push-based alerts, and support for FHIR Bulk Data export.

VII. CONCLUSION

EmberDB shows that high ingest throughput and native clinical semantics are not mutually exclusive. Encoding patient identity and LOINC codes into the storage key, partitioning the data into time-aligned chunks, and integrating the pattern-detection algorithms directly yields a system that handles continuous patient monitoring without giving up time-series performance. On the synthetic MIMIC-IV workload it ingests $3.8\text{--}9.4\times$ faster than SQLite, TimescaleDB, and InfluxDB and answers patient-scoped point queries in single-digit microseconds, while preserving FHIR structure with no post-hoc transformation. It is not uniformly faster: TimescaleDB matches it on cohort scans and InfluxDB stores the data in a fraction of the space, both of which point to the columnar chunk format as the next step. These results come from a single synthetic scale on one host, measured as means without latency distributions and through engine-specific client paths, so we present the ordering as a hypothesis: confirming it calls for a second scale, a real-data run, the analytical query shapes the current mix omits, and full latency distributions over an equal iteration budget.

AI-USAGE DISCLOSURE

The author used a large language model (Anthropic Claude) as a coding and writing assistant. Its role covered scaffolding and debugging the Rust benchmark harnesses (including the InfluxDB and TimescaleDB baseline drivers), drafting and editing prose, and formatting tables. All system design, experimental decisions, and benchmark execution were directed by the author, and every reported number was produced by running the code on the author’s hardware and verified against the committed CSV artifacts. The author takes full responsibility for the content.

REFERENCES

- [1] A. E. W. Johnson, T. J. Pollard, L. Shen, et al., “MIMIC-III, a freely accessible critical care database,” *Scientific Data*, vol. 3, p. 160035, 2016.
- [2] A. E. W. Johnson, L. Bulgarelli, L. Shen, et al., “MIMIC-IV, a freely accessible electronic health record dataset,” *Scientific Data*, vol. 10, p. 1, 2023.
- [3] HL7 International, “FHIR Release 4: Observation Resource,” <https://hl7.org/fhir/R4/observation.html>, 2019.
- [4] InfluxData, “InfluxDB: Open Source Time Series Database,” <https://www.influxdata.com/>, 2024.
- [5] Timescale Inc., “TimescaleDB: An Open-Source Time-Series SQL Database,” <https://www.timescale.com/>, 2024.